

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1402

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:100

Annexure A

EXPERIMENTS

Code of Conduct:

*I V.VAMSIDHAR REDDY(192365029) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
V.VAMSI

Annexure A

EXPERIMENTS

Q1. You are developing a compiler-based desk calculator that evaluates arithmetic expressions using syntax-directed definitions (SDD).

The system should:

- (i) Accept an arithmetic expression with operators +, -, *, / and parentheses
- (ii) Use syntax-directed evaluation to compute values based on operator precedence
- (iii) Optimize evaluation by reducing unnecessary temporary computations
- (iv) Display the final result of the expression

Demonstrate how SDD rules (synthesized attributes) are used for expression evaluation.

SDD

Grammar with semantic rules:

$$\begin{aligned} E &\rightarrow E + T \quad \{ E.val = E1.val + T.val \} \\ E &\rightarrow E - T \quad \{ E.val = E1.val - T.val \} \\ E &\rightarrow T \quad \{ E.val = T.val \} \end{aligned}$$
$$\begin{aligned} T &\rightarrow T * F \quad \{ T.val = T1.val * F.val \} \\ T &\rightarrow T / F \quad \{ T.val = T1.val / F.val \} \\ T &\rightarrow F \quad \{ T.val = F.val \} \end{aligned}$$
$$\begin{aligned} F &\rightarrow (E) \quad \{ F.val = E.val \\ &\} \\ F &\rightarrow \text{digit} \quad \{ F.val = \text{digit} \\ &\} \end{aligned}$$

Aim

To evaluate arithmetic expressions using Syntax-Directed Definition (SDD) and synthesized attributes.

Theory

SDD associates semantic rules with grammar productions. The val attribute is synthesized from the child nodes.

The grammar separates:

- $E \rightarrow \text{Expression}$
- $T \rightarrow \text{Term}$
- $F \rightarrow \text{Factor}$

This ensures * and / have higher precedence than + and -.

Example

For:

$2 + 3 * 4$

Evaluation:

$3 * 4 = 12$

$2 + 12 = 14$

Therefore:

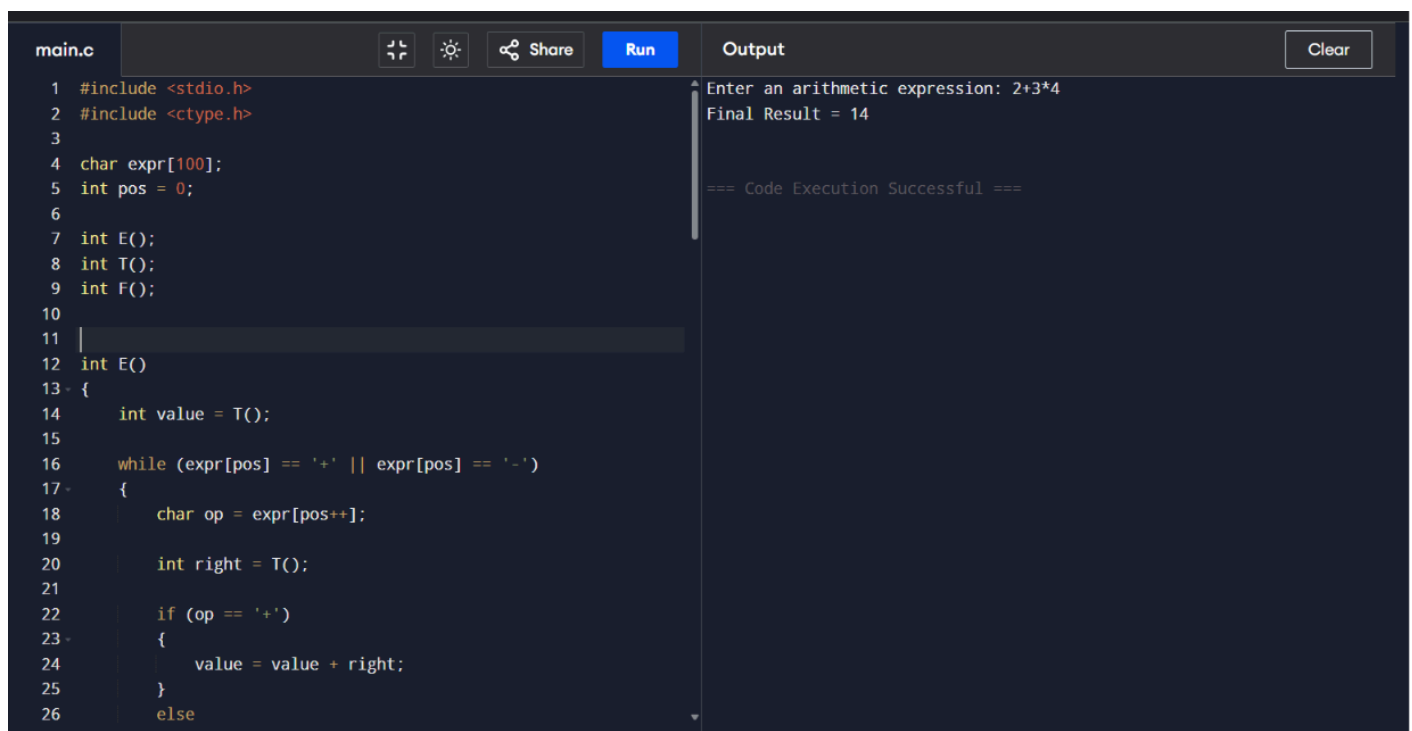
$E.val = 14$

Optimization

Intermediate values are directly passed between E, T, and F, reducing unnecessary temporary computations.

Result

Thus, the arithmetic expression is successfully evaluated using SDD and synthesized attributes.



```
main.c
1 #include <stdio.h>
2 #include <ctype.h>
3
4 char expr[100];
5 int pos = 0;
6
7 int E();
8 int T();
9 int F();
10
11
12 int E()
13 {
14     int value = T();
15
16     while (expr[pos] == '+' || expr[pos] == '-')
17     {
18         char op = expr[pos++];
19
20         int right = T();
21
22         if (op == '+')
23         {
24             value = value + right;
25         }
26         else
```

Output

Enter an arithmetic expression: 2+3*4
Final Result = 14

=== Code Execution Successful ===

Clear

Q2. In a compiler, postfix expressions are evaluated during code generation using stack-based bottom-up evaluation. Write a C program that:
Evaluates a postfix expression
Demonstrates S-attributed definition evaluation
Shows how intermediate results are computed

Aim

To evaluate a postfix expression using a **stack** and demonstrate S-attributed evaluation.

Theory

In postfix evaluation, operands are pushed onto a stack. When an operator is encountered, operands are popped, the operation is performed, and the result is pushed back.

Example

Postfix: 234*+

Evaluation:

$3 * 4 = 12$

$2 + 12 = 14$

Final result:

14

S-Attributed Evaluation

The intermediate result is synthesized from the operand values and passed upward for further computation.

Result

Thus, the postfix expression is successfully evaluated using **stack-based bottom-up evaluation** and S-attributed definition.

main.c

Share

Run

Output

Clear

```
1 #include <stdio.h>
2 #include <ctype.h>
3
4 #define MAX 100
5
6 int stack[MAX];
7 int top = -1;
8
9 void push(int value)
10 {
11     stack[++top] = value;
12 }
13
14 int pop()
15 {
16     return stack[top--];
17 }
18
19 int main()
20 {
21     char postfix[MAX];
22     int i;
23     int a, b, result;
24
25     printf("Enter postfix expression: ");
26     scanf("%s", postfix);
```

Enter postfix expression: 234*+
3 * 4 = 12
2 + 12 = 14
Final Result = 14

=== Code Execution Successful ===

Q3. You are designing a parser that uses L-attributed definitions, where inherited attributes are passed from parent to child.

Write a C program that:

- (i). Evaluates a simple expression like $a + b * c$
- (ii) Uses function parameters to simulate inherited attributes

Ensures correct precedence handling

Demonstrates top-down evaluation logic

Explain how inherited attributes influence computation.

Aim

To evaluate an expression using **L-attributed definitions** and function parameters to simulate inherited attributes.

Theory

In L-attributed definitions, information can be passed from a parent to its child. In C, function parameters can be used to simulate this flow of information.

Example

For:

$a + b * c$

If:

$a = 2$

$b = 3$

$c = 4$

Evaluation:

$b * c = 3 * 4 = 12$

$a + 12 = 14$

Inherited Attributes

Values are passed as function parameters from one evaluation level to another, representing inherited information.

Precedence

The grammar evaluates multiplication before addition, ensuring correct operator precedence.

Result

Thus, the expression is successfully evaluated using **L-attributed evaluation and inherited attributes**.

main.c

Share

Run

Output

Clear

```
1 #include <stdio.h>
2
3 int E(int a, int b, int c);
4 int T(int b, int c);
5 int F(int value);
6
7 int E(int a, int b, int c)
8 {
9     int tValue;
10    int result;
11
12    tValue = T(b, c);
13
14    result = a + tValue;
15
16    return result;
17 }
18
19 int T(int b, int c)
20 {
21     int first;
22     int second;
23     int result;
24
25     first = F(b);
26     second = F(c);
```

Enter value of a: 2
Enter value of b: 3
Enter value of c: 4

Expression: $a + b * c$
 $3 * 4 = 12$
 $2 + (3 * 4) = 14$
Final Result = 14

=== Code Execution Successful ===

Q4. In a compiler, it is important to check whether two type expressions are equivalent. Write a C program that:
(i). Accepts two type expressions (e.g., `int`, `float`, `int*`) (ii). Compares them for **type equivalence**
(iii). Displays whether they are:
(a). Equivalent (b). Not equivalent
Extend the logic to handle pointer types (`int*`, `float*`).

Aim

To compare two type expressions and determine whether they are equivalent.

Theory

Type equivalence checks whether two type expressions represent the same type.

Examples:

`int` and `int` → Equivalent

`int` and `float` → Not Equivalent

`int*` and `int*` → Equivalent

`int*` and `float*` → Not Equivalent

Pointer Types

For pointer types, both the pointer and its base type must match.

For example:

`int* ≠ float*`

because their base types are different.

Result

Thus, the program successfully checks equivalence between basic and pointer types.

main.c

Share

Run

Output

Clear

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int isEquivalent(char type1[], char type2[])
5 {
6     int pointer1 = 0;
7     int pointer2 = 0;
8     if (strchr(type1, '*') != NULL)
9         pointer1 = 1;
10    if (strchr(type2, '*') != NULL)
11        pointer2 = 1;
12    if (pointer1 != pointer2)
13        return 0;
14    if (strcmp(type1, type2) == 0)
15        return 1;
16    return 0;
17 }
18 int main()
19 {
20     char type1[20], type2[20];
21     printf("Enter first type: ");
22     scanf("%19s", type1);
23     printf("Enter second type: ");
24     scanf("%19s", type2);
25     if (isEquivalent(type1, type2))
26         printf("Equivalent\n");
```

Enter first type: int
Enter second type: int
Equivalent

=== Code Execution Successful ===

main.c

Share

Run

Output

Clear

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int isEquivalent(char type1[], char type2[])
5 {
6     int pointer1 = 0;
7     int pointer2 = 0;
8     if (strchr(type1, '*') != NULL)
9         pointer1 = 1;
10    if (strchr(type2, '*') != NULL)
11        pointer2 = 1;
12    if (pointer1 != pointer2)
13        return 0;
14    if (strcmp(type1, type2) == 0)
15        return 1;
16    return 0;
17 }
18 int main()
19 {
20     char type1[20], type2[20];
21     printf("Enter first type: ");
22     scanf("%19s", type1);
23     printf("Enter second type: ");
24     scanf("%19s", type2);
25     if (isEquivalent(type1, type2))
26         printf("Equivalent\n");
```

Enter first type: int
Enter second type: float
Not Equivalent

=== Code Execution Successful ===

Q5. You are building a semantic analyzer that detects type errors in arithmetic expressions. Write a C program that:

Accepts two operands and their types

Checks whether the operation (e.g., +, *) is valid

Reports:

(a). Valid expression

(b). Type error (e.g., `int + char*`)

Demonstrate how compilers detect semantic errors during type checking

Aim

To check whether an arithmetic operation is valid based on the types of its operands.

Theory

Type checking is performed during **semantic analysis**. It verifies whether operands are compatible with the given operator.

Examples:

`int + int` → Valid expression

`int + float` → Valid expression

`int + char*` → Type error

`int * int*` → Type error

Type Checking

The semantic analyzer checks the types of both operands before allowing the operation.

For example:

`int + char*`

is invalid because the operands are incompatible.

Result

Thus, the program successfully performs **semantic type checking** and identifies valid expressions and type errors.

main.c



Share

Run

Output

Clear

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int isValidOperation(char type1[], char operator, char type2[])
5 {
6
7     if (strchr(type1, '*') != NULL || strchr(type2, '*') !=
8         NULL)
9     {
10         return 0;
11     }
12     if ((strcmp(type1, "int") == 0 || strcmp(type1, "float") ==
13         0) &&
14         (strcmp(type2, "int") == 0 || strcmp(type2, "float") ==
15             0))
16     {
17         if (operator == '+' ||
18             operator == '-' ||
19             operator == '*' ||
20             operator == '/')
21         {
22             return 1;
23         }
24     }
25 }
```

Enter first operand: 10
Enter type of first operand: int
Enter operator (+, -, *, /): *
Enter second operand: 2.5
Enter type of second operand: float

Expression: 10 * 2.5
Valid expression

=== Code Execution Successful ===

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation